I have interviewed 100+ ML/AI engineers
I have never asked "explain transformers" in interviews.
Here's actually what I would ask:

🔹 **Scenario Questions**
1️⃣ "You inherit a RAG system. Users complain it's slow but accurate. How would you diagnose and improve it?"
↳ Looking for: Systematic approach, measurement before optimization, understanding trade-offs

2️⃣ "Your model works great Monday-Friday but performs poor on weekends. How would you investigate?"
↳ Looking for: Data distribution thinking, monitoring strategy, root cause analysis process

3️⃣ "You have $10K monthly budget for AI infrastructure. Design a recommendation system that scales."
↳ Looking for: Cost awareness, build vs buy decisions, incremental deployment strategy

🔹 **The Debugging Question**
"A production model suddenly drops from 95% to 60% accuracy. Walk me through your investigation."
Winners discuss:
→ Check data pipeline first, not model
→ Look for upstream changes
→ Verify monitoring wasn't broken
→ Compare distributions, not just accuracy
→ Have rollback ready before investigating

🔹 **The System Design Question**
"How would you build a system that summarizes customer support tickets in real-time?"
I'm not looking for "use GPT-5" - I want to hear:
→ How do you handle different ticket formats?
→ What's your approach to quality control?
→ How do you measure if summaries are helpful?
→ What happens when the LLM service is down?
→ How would you gather feedback and improve?

🔹 **The Trade-off Question**
"You can have fast, cheap, or accurate. Pick two and explain why."
The best answers:
✅ "Depends on the use case - let me give examples..."
✅ "Here's how I'd make that decision with stakeholders..."
✅ "Can we redefine 'accurate' for this problem?"
The worst: "I'd optimize for all three"

🔹 **The Coding Task**
"Here's a Jupyter notebook that works. How would you productionize it?"
I watch if they mention:
→ Error handling and logging
→ Configuration management
→ Testing strategy
→ Deployment approach
→ Monitoring plan
→ Documentation needs

What gets you hired:
Not knowing everything.
But knowing how to figure out anything.

Show me your thinking process.
Tell me about trade-offs.
Admit what you don't know.
Explain how you'd learn it.

The best engineers I've hired said:
"I haven't solved this exact problem, but here's how I'd approach it..."
Then they outlined a systematic plan that made sense.

Your homework:
→Pick any ML/AI system you use daily.
→Write a one-page doc on how you'd improve it.
→Include constraints, trade-offs, and success metrics.

That exercise teaches more than 10 courses.
What do you think?

♻️ Repost to help someone prep smarter
➕ Follow Shantanu for engineering lessons & real world
Join 19000+ *real-world DS/ML/AI builders here*: **https://lnkd.in/ds_SzEUH**